
Mastering System Design

Section 8: Performance - Concepts, Tools & Techniques

Performance

Hello and welcome to section 8 of our course and this section is all about performance.

In this section, we will dive into the heart of system performance.

Not just how fast a system runs, but how well it scales, responds and delivers under pressure.

[So let's get started.](#)

Performance- Section Agenda

1. Introduction to System Performance
2. Caching for Speed Optimization
3. Messaging & Queues for Decoupling
4. Concurrency & Parallelism
5. Database Performance Optimization Techniques
6. Summary and Recap - Performance

Welcome to the System Performance Section

In this section, we will cover **six core topics** that will sharpen your understanding of system performance from multiple angles:

Introduction to System Performance Defining what performance really means in system design — latency, throughput, SLAs, SLOs, and key metrics.

Caching Strategies How caching can dramatically reduce response time and backend load.

Asynchronous Communication Using queues and message brokers to improve scalability, fault tolerance, and overall performance.

Concurrency and I/O Models How modern systems handle multiple tasks efficiently using thread pools, non-blocking I/O, and related techniques.

Database Performance Optimization Techniques such as indexing, denormalization, partitioning,

sharding, and query optimization.

Summary & Best Practices Tying everything together with common interview scenarios and tips for designing systems that stay fast at scale.

Lecture 1: Introduction to System Performance

Let's get started with the first lecture — **Introduction to System Performance**.

Introduction to System Performance

Performance

Welcome & Lecture Overview

In this lecture, we will lay the foundation for everything else we will explore in the complete **System Performance** section.

We will learn what **performance** really means in the context of system design and why it is much more than just “making things faster.”

We'll break down key concepts such as:

- Latency vs Throughput
- Responsiveness vs Scalability
- How engineers measure performance in the real world using **SLAs**, **SLOs**, and percentiles

You will also understand why performance is not just a technical concern — it directly impacts **user experience**, **business metrics**, and **engineering priorities**.

Finally, we will touch on performance testing, monitoring, and how to think about performance holistically — not just as an optimization after the fact, but as a core part of system architecture itself.

Let's dive in.

What is Performance in System Design?

- Performance = How efficiently a system meets its functional requirements under load
- Key dimensions:
 - Speed: Time to respond
 - Capacity: Amount of work handled
 - Efficiency: Resource usage under load
- Performance is not a single metric — it's a multi-dimensional goal



What Exactly is Performance in System Design?

At its core, **performance** is about how efficiently a system runs and how well it meets its functional requirements — **especially under load**.

Performance is **not just about speed**. It is about how well the system holds up as demand grows, the number of users increases, or the data becomes larger.

Three Key Dimensions of Performance

Speed How quickly does the system respond to requests? This is typically measured in terms of **latency** (response time).

Capacity How much work can the system handle at once? This includes metrics like **throughput** (requests per second), the number of concurrent users it can support, and overall workload capacity.

Efficiency How efficiently does the system use available resources (CPU, memory, network, disk) under stress?

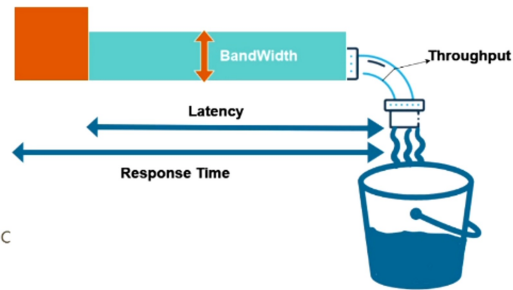
Important Insight

Performance is **not a single number or metric**. It is a **multidimensional goal**. As system designers, we must continuously balance **speed**, **scale (capacity)**, and **resource efficiency** to meet both technical requirements and business expectations.

A system that is fast but crashes under moderate load, or a system that is highly scalable but wastes resources, is not truly performant.

Latency vs. Throughput

- **Latency**
 - Time taken to process one request
 - Measured in ms or s
 - Affects responsiveness
- **Throughput**
 - Number of requests processed per sec
 - Measured in RPS or TPS
 - Affects scalability
- Low latency $\neq$ High throughput and vice versa
- Both must be balanced based on use case



Latency vs Throughput: Two Critical Performance Metrics

Two of the most important performance metrics in system design are **Latency** and **Throughput**. Understanding the difference between them is essential.

1. Latency

Latency is the time it takes to process a **single request** from start to finish.

- It is usually measured in **milliseconds (ms)** or seconds.
- It directly affects **responsiveness** — how fast the system *feels* to the end user.

Example: When a user clicks “Add to Cart,” the time from the click until the confirmation appears on the screen is the latency.

2. Throughput

Throughput is the number of requests a system can handle **per unit of time** (typically per second).

- It is often measured as **requests per second (RPS)**, **transactions per second (TPS)**, or operations per second.
- It reflects the **capacity** and **scalability** of the system.

Example: A system that can successfully process 10,000 checkout requests per second has high throughput.

Common Misconception

Many people assume that **low latency automatically means high throughput**.

This is **not always true**.

- You can have a very fast system (low latency) that can only handle a small number of users at a time.
- You can also have a high-throughput system that processes many requests per second but slows down individual requests (higher latency) under load.

The Real Challenge in System Design

The goal is to **balance both**:

- Keep **latency low** (good user experience)
- Keep **throughput high** (handle large scale)

The right balance depends on your application’s specific needs — whether you prioritize responsiveness (e.g., real-time apps) or raw capacity (e.g., batch processing systems).

Scalability vs. Responsiveness

- Scalability: Ability to handle increased load without performance degradation
 - Horizontal vs. Vertical scaling
- Responsiveness: System's ability to respond quickly
 - Tightly linked to latency
- Good design should ensure responsiveness at scale



Scalability vs Responsiveness

In system design, we often hear about **scalability** and **responsiveness**. While they are related, they are **not the same thing**.

1. Scalability

Scalability is the system's ability to handle **increased load** — more users, more data, or more requests — **without degrading performance**.

It answers the question: “**Can this system grow smoothly as demand increases?**”

We have already covered the two main approaches:

- **Vertical Scaling** (Scale Up) — Adding more resources to a single machine.
- **Horizontal Scaling** (Scale Out) — Adding more machines/instances.

2. Responsiveness

Responsiveness refers to **how quickly** the system responds to each individual request. It is tightly linked to **latency**.

Even under heavy load, a responsive system should still feel **snappy** and reactive to the user.

The Important Catch

A system can be **scalable** but **not responsive**. For example, it might successfully handle 10,000 concurrent users, but each user experiences several seconds of delay.

That is **not acceptable** for real-time or interactive applications.

The Real Goal in System Design

The ultimate objective is to achieve **responsiveness at scale**.

You want a system that can:

- Grow efficiently with increasing demand (**scalability**)
- Continue to deliver a great user experience (**low latency / high responsiveness**)

Balancing both is one of the most important challenges in modern system design.

Measuring Performance

- Performance must be measurable & trackable:
 - SLA (Service Level Agreement): External contractual guarantee
 - SLO (Service Level Objective): Internal target
 - SLI (Service Level Indicator): Actual metric value
- Example:
 - SLA: 99.9% uptime
 - SLO: 95% of requests < 300ms
 - SLI: Actual measurement (e.g., 93% of requests < 300ms)



Measuring Performance: SLAs, SLOs, and SLIs

To improve performance, we must first be able to **measure it** clearly, consistently, and accurately. This is where three important terms come in: **SLAs**, **SLOs**, and **SLIs**. These form the foundation of performance tracking in production systems.

1. SLA (Service Level Agreement)

An **SLA** is an **external contractual guarantee** made to users or clients.

- It defines the level of service the business promises to deliver.
- **Example:** “We guarantee 99.9% uptime every month.”

Failing to meet an SLA can have serious business consequences, including financial penalties, lost trust, or legal action.

2. SLO (Service Level Objective)

An **SLO** is an **internal performance target** set by the engineering team.

- It acts as a realistic goal that guides day-to-day operations.
- It is more flexible than an SLA.
- **Example:** “95% of all requests should complete under 300 milliseconds.”

SLOs help teams focus on what they aim to achieve internally.

3. SLI (Service Level Indicator)

An **SLI** is the **actual measured value** of a performance metric.

- It tells you what is really happening right now.
- **Example:** “Currently, only 93% of requests are completing under 300 milliseconds.”

This gap between SLI (93%) and SLO (95%) signals that investigation and optimization are needed.

Summary

- **SLA** → What you **promise** to the customer (external commitment).
- **SLO** → What you **aim for** internally (engineering target).
- **SLI** → What you are **actually achieving** right now (real measurement).

Together, these three concepts help teams monitor, analyze, and continuously improve system performance in a transparent and accountable way — both for the business and for users.

Understanding Percentiles

- Mean/average $\neq$ useful in tail-latency-sensitive systems
- Percentiles provide better insight:
 - P50: Median
 - P95: 95% of requests faster than this
 - P99: Tail latency — critical for user experience
- Track tail latencies for real-world performance insights

Understanding Percentiles in Performance Metrics

When it comes to performance metrics — especially **latency** — relying on averages (means) can be dangerously misleading.

Why? The average latency doesn't tell you how bad the **worst experiences** are. In real-world systems, it is often the **tail latency** (the slowest responses) that frustrates users and breaks experiences.

This is where **percentiles** become extremely valuable. They give you a much clearer picture of the **performance distribution**.

Key Percentiles You Should Know

- **P50 (Median)** 50% of the requests are faster than this value. It is a good general indicator, but **not sufficient** for high-performance systems.
- **P95** 95% of the requests are faster than this latency value. **Example:** If your P95 latency is 300 ms, it means 95% of requests complete in under 300 milliseconds.
- **P99 (Tail Latency)** 99% of the requests are faster than this value. This represents the slowest 1% of requests. These are often the **most important** to monitor, especially in real-time systems, interactive UIs, and customer-facing APIs.

Why Percentiles Matter More Than Averages

If you only optimize for the average, you are ignoring the **outliers** that cause real user pain.

Tracking **P95** and **P99** helps you design for **consistent performance** — not just “fast enough for most of the time,” but reliably fast for 99% of user requests.

In short: Always prefer percentiles over simple averages when performance really matters.

Why Performance Matters in Modern Applications

- Users expect instantaneous responses (esp. mobile/web)
- Poor performance leads to:
 - Drop-offs & bounce rates
 - Loss in revenue
 - System instability under load
- Performance is a feature, not an afterthought
- Impacts user experience, cost, and reputation

Why Performance Matters: It's a Core Feature, Not an Afterthought

In today's world, **performance is not optional** — it is a **core feature** of any successful application.

Why Performance is Critical

- **User Expectations** Users on mobile and web platforms expect **instantaneous responses**. A difference of just a few hundred milliseconds can be the line between a seamless experience and a frustrating one.
- **Business Impact** Poor performance has real consequences:
 - Users abandon slow websites or apps.
 - In e-commerce, every extra second of delay can result in significant lost revenue.
 - Under high load, slow performance directly hurts the bottom line.
- **Reputation Damage** If your application is slow, users won't just leave — they will **remember** the bad experience and are unlikely to return.

Key Mindset Shift

Performance should be treated as a **first-class citizen** in system design — not something you patch up later.

A fast and responsive system delivers:

- Higher user satisfaction
- Better operational efficiency
- Lower infrastructure costs (optimized systems waste fewer resources)

Bottom Line: Don't treat performance as an afterthought. In modern applications, **performance must be designed in from the start**.

Performance Testing Overview

- Types of performance testing:
 - Load Testing: Normal load conditions
 - Stress Testing: Beyond normal limits
 - Spike Testing: Sudden large load
 - Endurance Testing: Over extended time
- Goals:
 - Identify bottlenecks
 - Ensure reliability under real-world scenarios